

Buổi 3 — Tóm tắt: Dữ liệu thời gian với pandas, polars, DuckDB

Mục tiêu đầu bài. Sau buổi này bạn:

1. Đổi giờ địa phương sang **UTC** bằng tay và bằng pandas; chỉ ra giờ "không tồn tại" và giờ "xảy ra hai lần" ở New York.
2. Chỉ ra bằng số trên dữ liệu taxi thật: ngày đổi **giờ mùa hè** làm hỏng chuỗi đếm theo giờ ra sao.
3. Gộp đúng khi đổi tần suất: **cộng** cho số lượng, **trung bình** hoặc **giá trị cuối** cho số đo trạng thái; phân biệt giờ trống là "0" hay "không biết".
4. Ghép hai nguồn khác múi giờ mà không tạo quan hệ giả, và phát hiện ghép lệch bằng một phép thử nhanh.
5. Viết hàm `chuan_hoa_thoi_gian()` trả bảng **dạng dài** `unique_id, ds, y` theo UTC, đủ mốc, không trùng (**cột mốc M0** của khoá).

Dữ liệu lab: taxi vàng New York (NYC TLC) tháng 3/2024 (3.582.628 chuyến) và 11/2024 (3.646.369 chuyến); thời tiết New York theo giờ, UTC (Open-Meteo).

1. UTC, múi giờ và giờ mùa hè

Kết luận của phần. Một con số giờ không kèm múi giờ chưa chỉ ra một thời điểm. Đổi mọi dữ liệu về UTC có ghi múi giờ trước khi làm bất cứ việc gì khác.

UTC (Coordinated Universal Time)

- **Định nghĩa.** Giờ phối hợp quốc tế: một đồng hồ chung cho cả thế giới, chạy đều, không bao giờ vặn theo mùa. Hậu tố **Z** (`07:30Z`) nghĩa là giờ UTC, tương đương `+00:00`.
- **Ví dụ.** 07:00 sáng ở Hà Nội là 00:00Z.
- **Tính chất.** Mỗi giờ UTC ứng với **đúng một** thời điểm. Giờ địa phương thì không (xem giờ mùa hè).
- **Vai trò trong dự báo.** Là trục thời gian chung để gộp, ghép và chia dữ liệu mà không lệch.

Offset và múi giờ (time zone, IANA)

- **Định nghĩa.**
 - **Offset:** số giờ giờ địa phương lệch so với UTC.
 - **Múi giờ:** tên một vùng dùng chung luật giờ, ví dụ `America/New_York`. Bảng tra múi giờ mà mọi máy tính dùng do **IANA** giữ.
- **Công thức.**

$$t_{\text{UTC}} = t_{\text{địa phương}} - \text{offset}(t)$$

- **Ví dụ.**
 - Hà Nội offset +7: 07:15 $\rightarrow$ 00:15Z.
 - New York offset -5: 20:00 ngày 9/3 $\rightarrow$ 20:00 + 5 = 01:00Z ngày 10/3.
- **Tính chất.** Offset **phụ thuộc thời điểm**: một múi giờ có thể có nhiều offset trong năm. `Asia/Ho_Chi_Minh` luôn +7; `America/New_York` lúc -5, lúc -4.
- **Phân biệt.** Múi giờ là **luật** (tên vùng). Offset là **con số** tại một thời điểm. Lưu offset cố định thay cho múi giờ sẽ sai khi luật đổi.

Giờ mùa hè (daylight saving time, DST): EST và EDT

- **Định nghĩa.** Một số nơi vận đồng hồ nhanh 1 giờ vào mùa hè rồi vận lại vào mùa thu. Miền Đông nước Mỹ: **EST** (giờ chuẩn, UTC-5) và **EDT** (giờ mùa hè, UTC-4).
- **Ví dụ.** New York năm 2024:

Ngày	Việc xảy ra	Hệ quả
Chủ nhật 10/3	02:00 EST vận lên 03:00 EDT	02:00–02:59 không tồn tại ; ngày chỉ có 23 giờ
Chủ nhật 3/11	02:00 EDT vận lùi về 01:00 EST	01:00–01:59 xảy ra hai lần ; ngày có 25 giờ

- **Cách diễn giải.** Một giờ New York không kèm offset ứng với **0, 1 hoặc 2** thời điểm UTC:
 - 02:30 ngày 10/3 ứng với 0 thời điểm;
 - 01:30 ngày 3/11 ứng với 2 thời điểm: 05:30Z (EDT) và 06:30Z (EST).
- **Phân biệt.** Việt Nam không có giờ mùa hè. Độ lệch New York–UTC là 4 giờ **chỉ** từ 10/3 đến 3/11/2024, ngoài khoảng đó là 5 giờ.

Thời gian naive và aware

- **Định nghĩa.**
 - **Naive:** thời điểm không ghi múi giờ, ví dụ `2024-03-10 01:30`.
 - **Aware:** thời điểm có ghi múi giờ, ví dụ `2024-03-10 01:30-05:00`.
- **Tính chất.** Giá trị naive không xác định được thời điểm thật cho đến khi biết nó thuộc múi giờ nào.
- **Vai trò trong dự báo.** Mọi phép gộp, ghép, chia tập huấn luyện/kiểm tra phải làm trên thời gian aware ở UTC.

Gắn và đổi múi giờ trong pandas — phương pháp

- **Mục đích.** Đưa cột thời gian về UTC aware.
- **Quy trình.**
 1. `tz_localize("America/New_York")` : **gắn** múi giờ cho giá trị naive. Con số giờ giữ nguyên, giá trị thành aware.
 2. `tz_convert("UTC")` : **đổi** giá trị aware sang múi giờ khác. Con số giờ đổi, thời điểm thật giữ nguyên.
 3. Cột đã có offset (`+07:00`, `Z`) : dùng `pd.to_datetime(..., utc=True)`.
- **Xử lý giờ đặc biệt.** Mặc định `tz_localize` **báo lỗi** (`ValueError`). Hai tham số:
 - `nonexistent` (giờ không tồn tại, nonexistent time): `"raise"`, `"NaT"`, hoặc `"shift_forward"` (dời lên giờ hợp lệ kế tiếp).
 - `ambiguous` (giờ mơ hồ, ambiguous time — tức giờ lặp): `"raise"`, `"NaT"`, `"infer"` (đoán theo thứ tự dòng), hoặc mảng `True/False` (`True` là giờ mùa hè).
- **Phân biệt.** Nhầm `tz_localize` với `tz_convert` là lỗi hay gặp: gắn nhầm thì thời điểm sai, đổi nhầm thì báo lỗi với dữ liệu naive.

2. Giờ mùa hè làm hỏng dữ liệu taxi thật

Kết luận của phần. Cột giờ đón của taxi New York là giờ địa phương naive. Đếm trực tiếp trên nó tạo **một giờ 0** chuyển giao ngày 10/3/2024 và **một giờ gấp đôi** ngày 3/11/2024.

Giờ không tồn tại (nonexistent time)

- **Định nghĩa.** Giờ đồng hồ bị nhảy qua khi vận sang giờ mùa hè.
- **Ví dụ.** Bốn chuyến ghi 01:20, 01:50, 03:10, 03:40 ngày 10/3.

- Đếm trên giờ naive: 01:00 có 2, 02:00 có **0**, 03:00 có 2.
- Đổi sang UTC trước: 06Z có 2, 07Z có 2. Hai giờ liền nhau, không có giờ 0.
- **Tác động.** Chuỗi đếm có một giờ 0 chuyển giả. Tính thời lượng chuyển trên giờ naive thì **dur đúng một giờ**: có 1.093 chuyển như vậy trong tệp thật.

Giờ lặp / giờ mơ hồ (ambiguous time)

- **Định nghĩa.** Giờ đồng hồ xảy ra hai lần khi vặn lùi về giờ chuẩn.
- **Ví dụ.** Ngày 3/11, giờ 01:00 trên giờ naive có **9.869** chuyển, trong khi cùng giờ Chủ nhật tuần sau chỉ có **5.318**: hai giờ thật đã gộp làm một.
- **Tác động.**
 - Không thể tách chuyển nào thuộc EDT, chuyển nào thuộc EST.
 - Thời lượng tính trên giờ naive có thể **âm** (khách "được trả trước khi đón"). Tháng 11 có 1.078 chuyển như vậy, trong đó 1.001 chuyển đón lúc 01:xx ngày 3/11.
- **Cách xử lý.** Bỏ các dòng mơ hồ, ghi số dòng bỏ vào `df.attrs`, và ghi **NaN** cho cả hai mốc UTC chúng có thể thuộc về (05:00Z, 06:00Z), thay vì một con số bị đếm thiếu.

Chẩn đoán một cột thời gian — phương pháp

- **Mục đích.** Xác định múi giờ thật của một cột khi tài liệu nguồn không ghi.
- **Quy trình.** Hỏi năm câu:
 1. Kiểu dữ liệu và múi giờ (naive hay aware)?
 2. Khoảng thời gian từ đâu đến đâu?
 3. Có bao nhiêu dòng nằm ngoài kỳ dữ liệu?
 4. Có bao nhiêu dòng thời lượng âm?
 5. Mỗi ngày có bao nhiêu giờ có dữ liệu?
- **Ví dụ.** Tháng 3, câu 5 cho `{24: 30, 23: 1}`: ngày 23 giờ là 10/3, lộ ngay. Tháng 11, câu 5 cho `{24: 30}`: ngày 25 giờ không để lại dấu hiệu, chỉ lộ qua câu 4 hoặc số chuyển bất thường.
- **Khi nào hỏng.** Giờ lặp không làm đổi số giờ mỗi ngày trên giờ naive, nên chỉ đếm số giờ thì không phát hiện được.

3. Đổi tần suất (resampling)

Kết luận của phần. Khi đổi tần suất phải trả lời ba câu: mỗi sự kiện thuộc khoảng nào, gộp bằng hàm gì, giờ trống ghi gì. Số lượng thì cộng, trạng thái thì trung bình hoặc giá trị cuối. Giờ trống của đếm sự kiện là 0; của số đo là NaN.

Tần suất, chuỗi đều và chuỗi không đều

- **Định nghĩa.**
 - **Tần suất** (frequency): khoảng cách giữa hai mốc liền nhau của chuỗi.
 - **Chuỗi đều** (regular): các mốc cách đều nhau.
 - **Chuỗi không đều** (irregular): mốc lúc dày lúc thưa.
- **Ví dụ.** Chuyển taxi đến lúc 00:10, 00:50, 02:20 là chuỗi không đều.
- **Vai trò trong dự báo.** Mô hình dự báo cần chuỗi đều, nên chuỗi sự kiện phải được gộp lên một tần suất cố định.

Khoảng gộp: `closed` và `label`

- **Định nghĩa.**
 - `closed`: đầu nào của khoảng được tính vào. [9:00, 10:00) là **đóng trái** (có 9:00, không có 10:00).
 - `label`: khoảng được đặt tên bằng mốc đầu hay mốc cuối.

- **Công thức** (đóng trái, nhãn trái):

$$\text{nhãn}(t) = \lfloor t/\Delta \rfloor \times \Delta$$

Nghĩa là làm tròn **xuống** tới bội số của tần suất Δ . Trong pandas: `dt.floor("h")`.

- **Ví dụ.** Chuyển lúc 9:00, 9:40, 10:00, 10:20.
 - Đóng trái, nhãn trái: {9:00: 2, 10:00: 2}.
 - Đóng phải, nhãn phải: {9:00: 1, 10:00: 2, 11:00: 1}.
- **Tính chất.** Mặc định pandas khác nhau theo tần suất:
 - `h`, `D`: đóng trái, nhãn trái.
 - `W`, `ME`, `QE`, `YE`: **đóng phải, nhãn phải** (doanh số tháng 1 mang nhãn 31/1).
- **Vai trò trong dự báo.** Nhãn đầu khoảng chứa số liệu chỉ biết đủ ở **cuối** khoảng. Dùng nhãn 09:00 làm đầu vào lúc 9:05 là **rò rỉ tương lai** (data leakage): mô hình được xem thông tin chưa có tại thời điểm dự báo.

Hàm gộp (aggregation function)

- **Định nghĩa.** Cách biến nhiều giá trị trong một khoảng thành một con số.
- **Quy tắc chọn.**

Loại đại lượng	Ví dụ	Hàm gộp	Giờ trống ghi
số lượng, sự kiện (count / flow)	số chuyển, kWh	<code>sum</code> , <code>count</code>	0
số đo trạng thái (state / stock)	hiệu suất, giá	<code>mean</code> , <code>last</code>	NaN

- **Phân biệt.**
 - Cộng 60 lần đo nhiệt độ trong một giờ ra "1.500 độ": vô nghĩa.
 - Lấy trung bình số chuyển ra "số chuyển trung bình mỗi phút": đã đổi đơn vị mà không ai hay.

Giá trị thiếu: 0 và NaN (missing value)

- **Định nghĩa.**
 - **0**: đã quan sát, và giá trị bằng 0 (giờ đó không có chuyển nào).
 - **NaN** (Not a Number) / **NaT** (Not a Time): không biết giá trị.
- **Ví dụ.** Sự kiện 00:10 = 1, 00:50 = 2, 02:20 = 5:

Cách	Giá trị tại 00:00, 01:00, 02:00	Dùng khi
<code>resample("h").sum()</code>	3; 0; 5	đếm sự kiện
<code>resample("h").mean()</code>	1,5; NaN; 5	số đo trạng thái
<code>asfreq("h")</code>	mốc 00:10, 01:10, 02:10 — 1; NaN; NaN	không hợp với sự kiện

- **Tính chất.**
 - `asfreq` **không gộp**: nó chỉ lấy giá trị đúng tại mốc, và lưới bắt đầu từ sự kiện đầu.
 - `resample` chỉ phủ từ sự kiện đầu tới sự kiện cuối. Muốn đủ mốc cho cả kỳ thì `reindex` thêm.
- **Phân biệt.** Điền 0 cho giờ "không biết" là bịa ra một giờ vắng khách. "0 độ" là một nhiệt độ thật, khác "không đo".

Đổi tần suất — phương pháp

- **Mục đích.** Biến chuỗi sự kiện hoặc chuỗi đo dày thành chuỗi đều ở tần suất mong muốn.
- **Quy trình.**
 1. Đưa thời gian về UTC aware.

2. Làm tròn xuống theo tần suất: `dt.floor(tan_suat)`.
 3. Gộp theo hàm phù hợp loại đại lượng.
 4. `reindex` lên lưới đủ mốc, điền 0 hoặc NaN.
- **Ví dụ.** Tháng 3/2024 trên giờ naive, `resample` ra 744 giờ. Trên UTC chỉ ra **743**, vì ngày 10/3 ngắn một giờ.
 - **Khi nào hỏng.** Gộp trên giờ naive: thừa hoặc thiếu giờ ở hai ngày đổi giờ: `groupby` thay `resample`: không tự thêm giờ trống, phải `reindex` sau.

4. Ghép hai nguồn dữ liệu

Kết luận của phần. Đưa mọi nguồn về UTC có ghi múi giờ rồi mới ghép. Ghép lệch múi giờ không báo lỗi mà âm thầm dời dữ liệu, có thể **đổi dấu tương quan**. Ghép theo thời gian gần nhất thì chỉ được nhìn về quá khứ.

Ghép theo thời gian (temporal join) — phương pháp

- **Mục đích.** Gắn biến giải thích (ví dụ thời tiết) vào chuỗi cần dự báo (số chuyển) theo đúng thời điểm.
- **Quy trình.** Cả hai bảng về UTC aware, rồi `merge` trên cột thời gian. Hàm ghép nên **từ chối** thời gian naive (báo `ValueError`).
- **Ví dụ lỗi.** Code đầu buổi nói dòng taxi "15:00" (giờ New York) với dòng thời tiết "15:00" (UTC). Ngày 15/3 New York đang EDT, nên đường thời tiết bị **đẩy sang phải 4 giờ**: dòng taxi 15:00 nhận thời tiết của 11:00.
- **Tác động.** Tương quan mưa – số chuyển (tháng 3/2024, đã khứ mùa vụ):

Cách ghép	<i>r</i>
lệch (giờ New York naive với giờ UTC)	-0,100
đúng (cả hai về UTC)	+0,136

Cả hai đều yếu, nhưng **khác dấu**: mô hình học từ bảng ghép lệch sẽ tin rằng mưa làm giảm khách.

- **Khi nào hỏng.** Hai nguồn khác múi giờ, hoặc một nguồn tải về với offset cố định. Open-Meteo khi xin `timezone=America/New_York` ghi một offset (-04:00) cho cả khoảng, nên lệch 1 giờ vào mùa đông. Luôn tải UTC.

Phép thử giờ nóng nhất — phương pháp

- **Mục đích.** Phát hiện nhanh việc ghép lệch múi giờ.
- **Quy trình.** Tính nhiệt độ trung bình theo giờ trong ngày trên bảng đã ghép, rồi xem đỉnh rơi vào giờ nào.
- **Ví dụ.** Ghép đúng: nóng nhất lúc **16 giờ**. Code đầu buổi: `gio_nong_nhat()` trả **20**, tức New York "nóng nhất lúc 8 giờ tối", vô lý dễ thấy.
- **Ưu điểm.** Rẻ, không cần kiến thức chuyên môn: ai cũng biết buổi chiều nóng hơn buổi tối.

`merge_asof` (as-of join) — phương pháp

- **Mục đích.** Ghép khi bảng bên phải không có đúng mốc của bảng bên trái, ví dụ giá chỉ được ghi khi thay đổi.
- **Quy trình.** Mỗi dòng bên trái lấy dòng bên phải **gần nhất theo thời gian**, theo hướng của tham số `direction`.
- **Ví dụ.** Giá ghi lúc 09:30 = 9, 10:30 = 10, 11:00 = 11:

Dòng trái	"backward" (mặc định)	"forward"	<code>allow_exact_matches=False</code>
10:00	9	10	9
11:00	11	11	10

- **Giải định.** Giá trị đã có tại thời điểm ghép.
- **Khi nào hỏng.**
 - "forward" kéo giá trị tương lai (10:30) về dòng 10:00: rò rỉ tương lai.
 - Số liệu công bố trễ: dùng `allow_exact_matches=False` để không lấy dòng đúng cùng thời điểm.

Khử mùa vụ trước khi tính tương quan (mượn trước, buổi 6 học kỹ)

- **Định nghĩa.** Lấy giá trị trừ đi mức trung bình của **cùng giờ, cùng thứ**. Phần còn lại cho biết giờ đó đông hay vắng hơn bình thường.
- **Ví dụ.** 8 giờ sáng thứ Hai trung bình 5.000 chuyến, hôm nay 5.400 → +400.
- **Vai trò.** Nếu không khử, tương quan với mưa bị nhিপ ngày đêm chi phối.

5. Dạng dài và hàm chuẩn hoá thời gian (cột mốc M0)

Kết luận của phần. Dữ liệu nhiều chuỗi lưu ở dạng dài `unique_id, ds, y`: mỗi (chuỗi, mốc) một dòng. Mọi chuỗi trải lên cùng một lưới mốc UTC. Tháng 3/2024 theo UTC có **743** giờ, không phải 744.

Định dạng dài và định dạng rộng (long / wide format)

- **Định nghĩa.**
 - **Dạng dài:** mỗi dòng là một bộ (chuỗi, mốc, giá trị).
 - **Dạng rộng:** mỗi chuỗi là một cột.
- **Ví dụ.** 2 khu vực × 3 giờ: dạng dài 6 dòng; dạng rộng 3 dòng × 2 cột.
- **Tính chất.** Thêm chuỗi thì dạng dài chỉ thêm dòng, không thêm cột. `pivot` đổi dài sang rộng.
- **Vai trò trong dự báo.** Các thư viện Nixtla (statsforecast, mlforecast) ở các buổi sau nhận dạng dài.

Ba cột `unique_id, ds, y`

- **Định nghĩa.**
 - `unique_id`: tên chuỗi.
 - `ds` (datestamp): thời điểm, UTC aware.
 - `y`: giá trị cần dự báo.
- **Ví dụ.** `("161", 2024-03-01 05:00Z, 412)`.

Lưới chung (common time grid)

- **Định nghĩa.** Mọi chuỗi có cùng một danh sách mốc thời gian. Mốc không có dữ liệu vẫn có dòng ($y = 0$ với sự kiện).
- **Ví dụ.** Lưới tháng 3/2024 là khoảng nửa mở [05:00Z ngày 1/3; 04:00Z ngày 1/4):
 - 00:00 ngày 1/3 New York (EST) là 05:00Z; 00:00 ngày 1/4 (EDT) là 04:00Z.
 - **Có mốc đầu, không có mốc cuối**, nên có 743 mốc. Tính cả mốc cuối thì 744, dư một giờ của tháng 4.
- **Vai trò trong dự báo.** Cho phép **backtest** cùng mốc cắt cho mọi chuỗi. Backtest là giả vờ đứng ở nhiều mốc cắt trong quá khứ, dự báo, rồi so với điều đã xảy ra.
- **Dữ liệu thật.** 259 khu vực, 192.437 dòng, **53,7%** số ô bằng 0. Vài khu vực rất đông (Midtown Center 163.267 chuyến), phần lớn rất thưa.

Hàm `chuan_hoa_thoi_gian()` — phương pháp

- **Mục đích.** Biến một bảng thô có cột thời gian thành dạng dài chuẩn cho mọi buổi sau.
- **Quy trình.**

1. Mọi đầu vào về UTC aware. Cột naive: `tz_localize(mui_gio_nguon, ambiguous=mo_ho, nonexistent="NaT")` rồi `tz_convert("UTC")`.
 2. Bỏ dòng trùng **chỉ** với dữ liệu trạng thái (hoặc khi `bo_trung=True`).
 3. `dt.floor(tan_suat)`, rồi gộp theo `gop`.
 4. `reindex` mỗi chuỗi lên lưới đủ mốc; mốc mà dòng mơ hồ có thể thuộc về thì ghi NaN.
- **Kết quả.** Bảng `unique_id, ds, y`, UTC aware, đủ mốc, không trùng. `df.attrs` ghi số dòng bỏ vì giờ mùa hè và vì trùng.
 - **Giả định.** Biết múi giờ nguồn của cột naive.
 - **Khi nào hỏng.** Bật bỏ trùng trên dữ liệu sự kiện: hai chuyến cùng giây là hai chuyến thật, bỏ trùng sẽ xóa hơn 1,8 triệu chuyến. Kiểm tra: tổng số chuyến tháng 3 phải còn đúng 3.582.605.

Công cụ: pandas, polars, DuckDB

Parquet

- **Định nghĩa.** Định dạng tệp lưu bằng **theo cột** (columnar), có nén.
- **Tính chất.** Đọc một cột thì bỏ qua được phần lớn tệp. Chỉ đọc cột cần đã nhanh gấp 2,4 lần đọc mọi cột.

polars và DuckDB

- **Định nghĩa.**
 - **polars:** thư viện bảng dữ liệu, chạy được chế độ **lazy** (viết cả chuỗi lệnh trước, polars tối ưu rồi mới chạy). pandas chạy **eager** (từng lệnh một).
 - **DuckDB:** cơ sở dữ liệu chạy SQL trực tiếp trên tệp.
- **Ưu điểm.** Với nhiều tệp lớn, `scan_parquet` của polars hay SQL của DuckDB chỉ đọc cột cần, nhanh và ít RAM hơn nạp hết vào pandas.
- **Phân biệt khi gặp giờ lặp** (đếm theo giờ UTC ngày 3/11/2024):

Công cụ	Cách xử lý giờ mơ hồ	Hệ quả
pandas (đáp án)	ghi NaN cho hai mốc 05:00Z và 06:00Z	thấy rõ chỗ không biết
polars (<code>ambiguous="null"</code>)	gom 9.869 chuyến vào nhóm <code>null</code>	thấy rõ
DuckDB	chọn EST: dồn cả vào 06:00Z, mất 05:00Z	không cảnh báo; bảng trông sạch nhất nhưng sai nhất

- **Lưu ý.** DuckDB lấy múi giờ mặc định theo hệ điều hành, nên chạy trên hai máy có thể ra kết quả khác nhau. Luôn đặt `SET TimeZone = 'UTC'` ở đầu script.